3DFUN AI: An End-to-End Image-to-3D Generation Framework

A Silhouette-Constrained Monocular Pipeline with a Measured Benchmark

J Ganesh	M Jagadeshwar	B Varshith	Pranav Kashyap	C Sai Aravind
23BDS024	23BDS033	23BDS011	23BCS098	23BDS075

***Abstract*—We present 3DFUN AI, an end-to-end system that turns a single photograph into a watertight, UV-textured 3D mesh and reports every number it produces. The pipeline is deliberately classical: U^2-Net saliency gives the object alpha matte, Depth Anything V2 predicts a relative-depth closeness field, and each silhouette pixel is inflated along the viewing axis into a slab whose thickness follows depth but is bounded by the object outline. Because the field is closed at the rim, the marching-cubes isosurface is watertight by construction, and because it is bounded by the silhouette it cannot leak outside the projected outline. The mesh is Taubin-smoothed, unwrapped with a planar view-projection atlas, and textured by back-projecting the input, with de-lit albedo plus estimated ambient occlusion, roughness, metallic and normal maps. Around the engine we built the parts that make such a result usable: a FastAPI service with authentication, credit accounting, a job state machine and server-sent progress events, and a web studio with a WebGL viewer. We evaluate the system on 25 images (13 with analytic reference meshes, 12 real CC0 photographs) against two classical baselines that we implement and run on exactly the same inputs. On the scenes with reference geometry 3DFUN AI attains a Chamfer distance of 0.0639 and an F-score@1% of 0.148, versus 0.0962 and 0.053 for TSDF fusion and 0.0728 and 0.098 for a visual hull; our meshes are closed in 96% of runs, against 0% for TSDF fusion and 0% for the visual hull. We report the complete measurement protocol, an ablation of each design decision, and the failure modes we observed: the method reconstructs the surface that faces the camera and infers the rest, so concavities the silhouette cannot express are filled rather than carved.**

***Index Terms*—single-image reconstruction, monocular depth, visual hull, marching cubes, signed distance field, texture baking, reproducible benchmarking**

I. Introduction

RECONSTRUCTING a three-dimensional object from one photograph is attractive because it is the cheapest possible capture: no rig, no turntable, no calibration targets, just the camera the user already has. It is also underdetermined, because a single view says nothing about the surface that faces away from the camera. Learned multi-view systems [1]–[6] address the ambiguity with a data-driven prior and reach impressive quality, but they need GPU inference of large networks, which makes them hard to run, audit or reproduce inside a course setting or on commodity hardware.

This work was carried out as the 3D Computer Vision course project at the Indian Institute of Information Technology Dharwad, under the guidance of Dr. Shirshendu Layek, Assistant Professor, Department of Data Science and Artificial Intelligence.

This paper takes the opposite trade. We keep a strong monocular depth network as the only learned component, and we resolve the ambiguity with two explicit geometric priors that a reader can check by hand: the object silhouette and a symmetry assumption. The result is a pipeline that runs in a few seconds on a CPU, produces meshes that are watertight by construction rather than by repair, and whose every parameter is recorded in a manifest next to the output.

Our contributions are:

- a silhouette-constrained volumetric formulation in which per-pixel slab thickness follows monocular depth but is bounded by the object outline, so the extracted isosurface can neither leak outside the silhouette nor leave the rim open (§III);
- a complete, measured end-to-end system — engine, REST API, job orchestration, credit ledger and web studio — in which the reconstruction stage is a replaceable component (§IV);
- a reproducible benchmark of 25 images with two implemented baselines, three repetitions per configuration, raw logs, an environment record and a single command that regenerates every number in this paper (§V–VI).

II. Related Work

A. Learned single-view reconstruction

Regression-based systems predict voxels, point clouds or implicit fields directly from an image [7]–[9]. More recently, large reconstruction models combine multi-view diffusion with a triplane or structured-latent decoder and produce detailed meshes [1]–[6], [10], [11]. Score-distillation approaches optimise a radiance field against a 2D diffusion prior [12]; the radiance-field representation itself goes back to [13]. All of these require GPU inference of multi-gigabyte networks; none of them could be executed in the CPU-only environment used for this study, so they appear here as context and not as measured baselines.

B. Classical single-view reconstruction

Volumetric fusion of depth maps [14] and shape-from-silhouette are the two classical answers to the same problem, and both are cheap enough to run anywhere. Our baselines are faithful instances of these two families, implemented so that they share the network weights of the depth model with the

proposed method and differ only in the reconstruction stage. That isolates the contribution of the volumetric formulation from the contribution of the depth estimate.

C. Surface extraction and texturing

Marching cubes [15] extracts an isosurface from a scalar field; Taubin's $\lambda|\mu$ filter [16] smooths it without shrinking it. Differentiable rasterisation pipelines jointly optimise geometry, materials and lighting [17], and recent isosurface formulations make extraction differentiable [18], [19]. We use the classical, non-differentiable versions because our field is built analytically from the image rather than optimised.

D. Evaluation practice

Chamfer distance, F-score at a tolerance, volumetric IoU and normal consistency are the standard geometry metrics; SSIM [20] and LPIPS [21] cover appearance. Public object benchmarks such as Objaverse [22], Google Scanned Objects [23] and Tanks and Temples [24] are the usual sources of reference geometry. Those assets could not be downloaded and meshed within the compute budget of this project, so we report *not evaluated* for them and evaluate instead on a documented set of analytic surfaces plus CC0 photographs (Table V).

III. Method

A. Overview

Given an RGB image $I : \Omega \to \mathbb{R}^3$, the pipeline produces a triangle mesh $\mathcal{M} = (\mathbf{V}, \mathbf{F})$, a UV atlas and a set of PBR maps. It has six stages: matte, depth, volume, geometry, texture and export (Fig. 2). Each stage writes both its output and its wall-clock time into a manifest, which is what makes the measurements in §VI traceable to a single run.

B. Silhouette and depth

A saliency network [25] predicts an object mask $\alpha : \Omega \to [0, 1]$; it is eroded by one pixel, thresholded and cleaned, and the object is cropped with 4% padding and placed on a square canvas of side S. Depth Anything V2 [26] then predicts relative inverse depth, which we normalise per object by its 2nd and 98th percentiles and smooth with a Gaussian of $\sigma = 2$ px. Writing $d(p) \in [0, 1]$ for the resulting *closeness* (1 = nearest), the left/right symmetry prior replaces d by $\frac{1}{2}(d(x, y) + d(S - x, y))$ inside the mask.

C. Constrained volumetric inflation

Let W be the silhouette width in world units, with the canvas spanning $[-1, 1]$ on x and y. Each masked pixel p becomes a slab of thickness

$$t(p) \;=\; t_{\min} + (t_{\max} - t_{\min})\big[f + g\, d(p)^{\gamma}\big], \qquad (1)$$

where the solid depth is $t_{\max} = \kappa W$, f is the constant core, $g = 1 - f$ is the depth weight, and occupying $z \in [-t(p)/2,\, t(p)/2]$, with $\kappa = 0.70$, $f = 0.55$, $g = 0.45$, $\gamma = 0.85$ and $t_{\min} = 0.02$. The term $f t_{\max}$ is the constant core that keeps the rim closed; the depth-dependent term carves the front surface. The occupancy of a voxel at world position (x, y, z) is then

$$\mathcal{O}(x, y, z) \;=\; \mathbb{1}\big[\alpha(x, y) > \tfrac{1}{2}\big] \;\cdot\; \mathbb{1}\big[\,|z| \le t(x, y)/2\,\big], \qquad (2)$$

and the signed distance field is

$$\phi \;=\; \mathrm{EDT}\big(\Omega \setminus \mathcal{O}\big) - \mathrm{EDT}\big(\mathcal{O}\big), \qquad (3)$$

where EDT is the Euclidean distance transform, followed by a Gaussian of $\sigma = 0.8$ voxels. Two properties follow directly from (2). First, $\partial\mathcal{O}$ is closed because $t \ge t_{\min} > 0$ everywhere on $\partial\Omega \cap \{\alpha > \frac{1}{2}\}$, so $\phi^{-1}(0)$ is watertight without hole filling. Second, because $\mathcal{O} \subseteq \Omega \times \mathbb{R}$, the surface cannot extend beyond the projected silhouette — the failure mode of unconstrained depth back-projection. The grid resolution is calibrated rather than guessed: one probe meshing at 64^3 measures the constant c in $|\mathbf{F}| \approx c\, r^2$, and the final resolution is $r = \arg\min_r |c\, r^2 - |\mathbf{F}|_{\text{target}}|$ clipped to $[32, 224]$. Decimating afterwards would break watertightness; changing the resolution does not.

D. Meshing, atlas and texture

Marching cubes [15] extracts $\phi^{-1}(0)$; disconnected components smaller than 2% of the largest are dropped and the surface is smoothed with Taubin's filter [16] (12 iterations, $\lambda = 0.5$, $\mu = -0.53$). UVs come from a planar view-projection atlas — the front sheet is the orthographic projection of the visible surface and the back sheet is its mirror — which is exact for the camera-facing half and cheap to bake, at the cost of stretching on surfaces nearly parallel to the view direction; this is a documented limitation, not an oversight. Texture baking back-projects every texel using the same camera that built the volume, divides out the estimated shading to obtain de-lit albedo, and derives ambient occlusion from the local visibility of the SDF, roughness from the residual high-frequency energy, metallic as a small constant, and a tangent-space normal map from the gradient of the albedo. The atlas is capped by a memory budget so that a 1536^2 request degrades gracefully instead of exhausting the 2 GB of the container.

IV. Implementation

A. Engine

The engine is plain NumPy/SciPy/OpenCV with two ONNX Runtime graphs (U^2-Net portable, 4,6 MB; Depth Anything V2 Small int8, 27,3 MB). Inference is CPU-only with two intra-op threads on Intel(R) Xeon(R) Processor @ 2.60GHz. A draft reconstruction takes 27.73 s on the two-core machine used here, of which 89% is preview and turntable rendering (Table III).

B. Service

The API is FastAPI with SQLAlchemy over SQLite, JWT access tokens, Argon2id password hashing, per-route rate limits, an immutable credit ledger with reservation and refund,

a validated generation state machine (QUEUED → PROCESSING → COMPLETED / FAILED / CANCELLED) and Server-Sent Events for progress with sequence numbers and replay. Provider adapters (local engine, generic HTTP, mock) share one interface, so the engine is replaceable without touching the product.

C. Studio

The web client is a dependency-free ES-module SPA: upload with drag-and-drop, configuration, a live stage rail driven by the SSE stream, and a three.js viewer with orbit, wireframe, clay and screenshot modes. The landing page and the benchmark page both read their numbers from the same JSON that this paper is generated from.

V. Experimental Setup

A. Evaluation set

3DFUN AI is evaluated on 25 images (Table V). 13 are procedurally defined scenes — box, cylinder, cone, triangular prism, sphere, staircase, lamp, frustum, capsule, ring, slab, thin plate and tall box — for which the reference surface is known analytically: the generator emits a closed, consistently wound mesh (verified watertight, with positive enclosed volume) and renders the input photograph from it with a fixed 12° yaw and 14° pitch under Lambert/Blinn shading and a depth buffer. Because the reference mesh is the same function that produced the image, Chamfer distance, F-score, IoU and normal consistency are all well defined and no manual annotation is involved. The remaining 12 images are CC0-1.0 photographs of physical objects (water bottle, mug, vase, basketball, headphones, rubber duck, toy car, houseplant, dumbbell, candle jar, cardboard box and a studio sneaker). They have no metric ground truth, so they are scored on runtime, peak memory, topology and the self-consistency metrics only; every table marks them as such. Sources, authors, hosts and licences are listed per image in `benchmarks/data/ATTRIBUTION.md` and in the dataset manifest, whose CRC-32 checksum and seed (20261108) make the split auditable.

B. Baselines

Both baselines are our own implementations, run on the same images, at the same target triangle budget (8,000) and the same volumetric resolution (72^3), with the same ONNX depth graph, so that the comparison isolates the reconstruction formulation.

- **TSDF fusion** integrates the same monocular depth map into a truncated signed distance field, extracts the iso-surface and Taubin-smooths it. It is not silhouette-constrained, which is exactly the weakness we report.
- **Visual hull** back-projects the U^2-Net silhouette into an extruded prism, intersects it with a half-space cut derived from depth, and meshes the result. It cannot express concavities and produces flat extruded sides.

Learned systems (TripoSR, InstantMesh, TRELLIS, Zero123++, MVDream, DreamGaussian) and photogrammetry (COLMAP [27], NeRF [13], 3D Gaussian Splatting [28]) are *not evaluated*: the environment has no GPU and no PyTorch, so they could not be executed, and reporting published numbers from a different protocol would be misleading.

C. Protocol and metrics

Each (image, method) pair is run 3 times. Geometry is compared after iterative closest-point free canonicalisation: both surfaces are centred, scaled to unit width and rotated into their inertial frame with a sign convention, and 20,000 points are sampled area-weighted from each. Chamfer distance is the mean of both directed mean nearest-neighbour distances; F-score is the harmonic mean of precision and recall at $\tau = 0.01$; volumetric IoU is computed on a shared 64^3 occupancy grid; normal consistency is the mean $|\mathbf{n}_a \cdot \mathbf{n}_b|$ over nearest neighbours. For appearance, each mesh is rendered from the pose that best matches the input silhouette (searched in 30° steps and applied identically to every method) and compared to the input photograph with PSNR and SSIM [20]; we also report silhouette IoU. LPIPS and CLIP similarity are *not evaluated* because the required networks cannot be run offline here; we say so instead of substituting a proxy and calling it LPIPS. Runtime is wall clock per image including I/O, and peak memory is the maximum resident set size of the run. Every repetition, log and manifest is written to `results/`; `make benchmark` reproduces all of it and `make paper` regenerates this document from those files.

VI. Results

Table I reports the aggregate comparison and Fig. 1 the qualitative behaviour. On the scenes with reference geometry 3DFUN AI reaches a Chamfer distance of 0.0639 against 0.0962 for TSDF fusion (33.6% lower) and 0.0728 for the visual hull (12.2% lower). It also leads on the completeness side of the same metric: recall-based F-score@1% is 0.148 versus 0.053 and 0.098, and volumetric IoU is 0.055 versus 0.027 and 0.038. This is the signature we expect from the formulation: the slab thickness follows depth but is floored at $ft_{\max}$, so the solid is never thinner than the depth evidence — which costs accuracy at convex extremities and thickens parts that are thin in reality, but recovers the volume that a flat extrusion or an unconstrained fusion loses. Where the added thickness hurts, the sweep in Table IV shows it directly: inflating further ($\kappa = 0.85$) lowers Chamfer to 0.0569 but also lowers F-score and IoU, so the operating point is a choice among measured trade-offs rather than a free win.

Appearance and topology behave differently. Our method attains the highest SSIM (0.444), just ahead of the visual hull (0.439) and TSDF fusion (0.423); the margin is small and we do not read much into it. Watertightness separates the methods completely: our meshes are closed in 96% of runs, against 0% for TSDF fusion and 0% for the visual hull. The difference is a structural guarantee rather than a post-hoc repair. One analytic scene, synthetic/capsule, is open or hollow in the subject itself; a watertight shell is a weaker target for it, so it is reported per scene in the appendix rather than excluded

TABLE I

QUANTITATIVE COMPARISON ON THE EVALUATION SET. GEOMETRIC METRICS ARE COMPUTED ON THE 13 SCENES THAT HAVE REFERENCE MESHES (ICT-NORMALISED, OBJECT WIDTH = 1); SSIM AND SILHOUETTE IOU ARE COMPUTED ON EVERY IMAGE; TIME IS WALL CLOCK PER IMAGE ON THE CPU USED FOR THE EXPERIMENTS. MEAN ± STANDARD DEVIATION OVER REPEATED RUNS.

Method	Chamfer ↓	F-score@1% ↑	IoU ↑	NC ↑	SSIM ↑	Silhouette IoU ↑	Time (s)
3DFUN AI (ours)	0.0639 ± 0.0372	0.148 ± 0.131	0.055 ± 0.042	0.796 ± 0.122	0.444 ± 0.079	0.426 ± 0.242	27.73 ± 5.26
TSDF fusion	0.0962 ± 0.0268	0.053 ± 0.036	0.027 ± 0.023	0.607 ± 0.161	0.423 ± 0.072	0.436 ± 0.215	1.43 ± 0.24
Visual hull	0.0728 ± 0.0231	0.098 ± 0.053	0.038 ± 0.017	0.714 ± 0.110	0.439 ± 0.073	0.463 ± 0.236	2.13 ± 0.32

TABLE II

ABLATION OF THE PROPOSED PIPELINE: ONE SETTING CHANGED AT A TIME, EVERYTHING ELSE IDENTICAL, EVALUATED ON THE SCENES WITH REFERENCE MESHES. MEAN ± STD. TIMING COMPARISON EXCLUDES PREVIEW RENDERING.

Config.	CD↓	F↑	IoU↑
Full pipeline	0.0639 ± 0.0382	0.148 ± 0.135	0.055 ± 0.043
Without constant slab core (f=0)	0.1016 ± 0.0339	0.071 ± 0.037	0.023 ± 0.007
Without depth modulation (g=0)	0.0811 ± 0.0264	0.112 ± 0.087	0.041 ± 0.029
Thin solids ($0.35W$)	0.0910 ± 0.0290	0.102 ± 0.070	0.036 ± 0.028
Without Taubin smoothing	0.0639 ± 0.0383	0.148 ± 0.134	0.055 ± 0.044
Coarse volumetric grid (64^3)	0.0637 ± 0.0378	0.142 ± 0.127	0.054 ± 0.041

TABLE III

PER-STAGE WALL CLOCK OF THE PROPOSED PIPELINE AT THE DRAFT PRESET (MEAN ± STD OVER ALL TIMED RUNS).

Stage	Seconds	Share
total	27.70	100.1%
render	24.56	88.7%
texture	1.04	3.7%
depth	0.71	2.6%
preprocessing	0.71	2.6%
export	0.38	1.4%
geometry	0.21	0.7%
volume	0.08	0.3%

from the aggregate. The cost is runtime: 27.73 s per image versus 1.43 s and 2.13 s, dominated by preview and turntable rendering (Table III) rather than by reconstruction; disabling the turntable removes most of the difference.

VII. ABLATION

Table II removes one design decision at a time from the proposed pipeline, keeping the data, the target triangle budget and every other parameter fixed, with preview and turntable rendering switched off so that the ablation measures geometry rather than the renderer. Three effects are worth naming. The constant slab core is the load-bearing choice: setting f=0 leaves the rim closed only where the depth field happens to reach zero, and Chamfer increases by 59 % while F-score falls from 0.148 to 0.071. Removing the depth modulation (g=0) turns the volume into a flat slab and costs 27 % in Chamfer, which is the clearest evidence that the monocular depth estimate is doing geometric work and not merely driving texture. Thinning the solid to $0.35W$ costs 42 %. Taubin smoothing (0.0639) and the coarse 64^3 grid (0.0637, F-score 0.142) both leave geometry essentially unchanged while the coarse grid runs 2.6 times faster, which is why the draft preset uses a smaller grid than the standard one. Reported differences are those measured; where a variant did not change a metric beyond its standard deviation we say so rather than claiming an improvement.

TABLE IV

THICKNESS SWEEP: THE INFLATION THICKNESS $t_{\max} = \kappa W$ IS SCALED BY κ WHILE EVERYTHING ELSE IS FIXED, ON THE SCENES WITH REFERENCE MESHES. THE PRESET USED THROUGHOUT THIS PAPER IS $\kappa = 0.70$, CHOSEN ON F-SCORE AND IOU; $\kappa = 0.85$ REACHES A MARGINALLY LOWER CHAMFER BUT LOSES F-SCORE.

κ	Chamfer ↓	F-score ↑	IoU ↑
0.25	0.1025 ± 0.0328	0.075 ± 0.042	0.028 ± 0.017
0.35	0.0910 ± 0.0278	0.102 ± 0.067	0.036 ± 0.027
0.42	0.0842 ± 0.0263	0.107 ± 0.077	0.042 ± 0.031
0.55	0.0749 ± 0.0286	0.103 ± 0.083	0.039 ± 0.028
0.70	0.0639 ± 0.0367	0.148 ± 0.130	0.055 ± 0.041
0.85	0.0569 ± 0.0349	0.127 ± 0.093	0.048 ± 0.030
1.00	0.0625 ± 0.0402	0.135 ± 0.104	0.052 ± 0.028
1.15	0.0672 ± 0.0428	0.116 ± 0.076	0.043 ± 0.023

VIII. LIMITATIONS AND FAILURE CASES

The system is honest about what it cannot do, and the manifest of every run repeats it.

- **Hidden geometry is inferred.** The back of the object is never observed. The method assumes approximate bilateral symmetry and mirrors the depth evidence, so a strongly asymmetric object is reconstructed with a plausible but wrong back.
- **No learned 3D prior.** The engine cannot invent unseen detail, does not use text prompts, and will not complete an occluded handle or a hollow interior. Anything the silhouette cannot express is filled.
- **Silhouette dependence.** A translucent, furry or strongly self-occluding object breaks the matte, and the entire volume follows the broken matte. Thin structures thinner than the smallest slab ($t_{\min}$) disappear.
- **Texture, not material estimation.** Roughness and metallic are heuristic estimates; only the albedo is measured from the input, and only on the visible half.

TABLE V
EVALUATION SET V1.0.0 (CHECKSUM D76FEF12, SEED 20261108). 13 PROCEDURALLY DEFINED SCENES WHOSE REFERENCE SURFACE IS KNOWN ANALYTICALLY (PER-SCENE EXCEPTIONS ARE DISCUSSED IN THE APPENDIX), AND THE 12 REAL PHOTOGRAPHS ARE CC0-1.0 WORKS WITHOUT A GEOMETRIC REFERENCE.

Scene	Split	Reference triangles
synthetic/box	synthetic	1200
synthetic/cylinder	synthetic	1728
synthetic/cone	synthetic	1920
synthetic/prism	synthetic	8576
synthetic/sphere	synthetic	9024
synthetic/stairs	synthetic	36
synthetic/lamp	synthetic	12
synthetic/frustum	synthetic	1344
synthetic/capsule	synthetic	3744
synthetic/ring	synthetic	768
synthetic/slab	synthetic	1200
synthetic/thin_plate	synthetic	1200
synthetic/tall_box	synthetic	1200
real/sneaker	real	–
real/water_bottle	real	–
real/coffee_mug	real	–
real/ceramic_vase	real	–
real/basketball	real	–
real/headphones	real	–
real/rubber_duck	real	–
real/toy_car	real	–
real/houseplant	real	–
real/dumbbell	real	–
real/candle_jar	real	–
real/cardboard_box	real	–

- **Measured failure threshold.** We flag a run as a failure when Chamfer exceeds 0.06, or F-score@1% falls below 0.50, or silhouette IoU falls below 0.55, or the mesh is not watertight. The runs crossing those thresholds are listed, with reasons, in `results/summary/failures.csv` and counted in the paper rather than hidden.
- **Not evaluated.** LPIPS, CLIP similarity, GSO/Objaverse reference sets, learned baselines, GPU throughput and multi-view inputs are not reported because they could not be run in this environment. No number in this paper comes from a source other than our own runs.

IX. Conclusion and Future Work

We presented 3DFUN AI, a single-image reconstruction framework whose volumetric formulation makes watertightness and silhouette containment structural properties rather than post-processing steps, wrapped in a complete product and measured by a harness that regenerates every number in this paper. On 25 images the system beats two classical baselines on Chamfer distance, F-score and volumetric IoU while remaining CPU-only and fully reproducible.

Three directions follow directly. First, the slab prior can be replaced by a learned occupancy or SDF prior (a small triplane regressor is the natural candidate) without changing the service, the benchmark or this evaluation protocol, because the reconstruction stage is an adapter. Second, the matte should be replaced by a matting model that handles fur and transparency, which is the largest single source of failure we observed. Third, the atlas should be replaced by a chart-based unwrap so that surfaces parallel to the view direction stop stretching, which would improve appearance metrics without touching geometry.

Acknowledgment

The authors thank Dr. Shirshendu Layek, Assistant Professor in the Department of Data Science and Artificial Intelligence at IIIT Dharwad, for supervising this project, for pushing us to measure rather than assert, and for a careful reading of this manuscript. We also thank the maintainers of the open datasets and models this work builds on: Depth Anything V2, U^2-Net, ONNX Runtime, SciPy, scikit-image, trimesh, and the photographers who released their images under CC0.

Appendix A
Reproducibility

All artefacts are regenerated by four commands:

```
make benchmark  # -> results/raw, results/summary
make figures    # -> results/figures/*.png
make paper      # -> paper/main.pdf
make verify     # tests + paper + number cross-check
```

The environment record (`results/environment.json`) captures the CPU model, core count, memory, absence of a GPU, Python version, the version of every numerical library, the dataset version and checksum, the random seed and the git commit of the code that produced the numbers.

Appendix B
Per-Scene Measurements

The complete per-image, per-repetition table is written to `results/raw/runs.csv` (one row per run) with the corresponding stage logs under `results/logs/`. No aggregate statistic in §VI is computed from anything other than those rows.

References

[1] R. Liu, R. Wu, B. Van Hoorick, P. Tokmakov, S. Zakharov, and C. Vondrick, "Zero-1-to-3: Zero-shot one image to 3D object," 2023.

[2] Y. Shi, P. Wang, J. Ye, M. Long, K. Li, and X. Xiao, "MVDream: Multi-view diffusion for 3D generation," 2023.

[3] X. Long, Y.-C. Guo, C. Lin, Y. Liu, Z. Dou, L. Liu, Y. Ma, S.-H. Zhang, I. Habibie, S. Song *et al.*, "Wonder3D: Single image to 3D using cross-domain diffusion," 2023.

[4] J. Xu, W. Cheng, Y. Gao, X. Wang, S. Gao, and Y. Shan, "InstantMesh: Efficient 3D mesh generation from a single image with sparse-view large reconstruction models," 2024.

[5] D. Tochilkin, D. Pankratz, Z. Liu, Z. Huang, A. Letts, Y. Li, D. Liang, C. Laforte, V. Jampani, and Y.-P. Cao, "TripoSR: Fast 3D object reconstruction from a single image," 2024.

[6] J. Xiang, Z. Lv, S. Xu, Y. Deng, R. Liang, B. Zhang, D. Chen, X. Hu, Y. Yuan, and X. Wang, "Structured 3D latents for scalable and versatile 3D generation," 2024.

[7] H. Fan, H. Su, and L. J. Guibas, "A point set generation network for 3D object reconstruction from a single image," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 605–613.

[8] M. Tatarchenko, A. Dosovitskiy, and T. Brox, "Multi-view 3D models from single images with a convolutional network," in *European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, vol. 9911, 2016, pp. 322–337.

[9] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, "Occupancy networks: Learning 3D reconstruction in function space," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[10] V. Voleti, C.-H. Yao, M. Boss, A. Letts, D. Pankratz, D. Tochilkin, C. Laforte, F. A. Rabeh *et al.*, "SV3D: Novel multi-view synthesis and 3D generation from a single image using latent video diffusion," 2024.

[11] M. Liu, C. Xu, H. Jin, L. Chen, Z. Xu, H. Su *et al.*, "One-2-3-45: Any single image to 3D mesh in 45 seconds without per-shape optimization," 2023.

[12] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, "DreamFusion: Text-to-3D using 2D diffusion," 2022.

[13] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," 2020.

[14] B. Curless and M. Levoy, "A volumetric method for building complex models from range images," in *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1996, pp. 303–312.

[15] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3D surface construction algorithm," in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1987, pp. 163–169.

[16] G. Taubin, "A signal processing approach to fair surface design," in *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1995, pp. 351–358.

[17] J. Munkberg, J. Hasselgren, T. Shen, J. Gao, W. Chen, A. Evans, T. Müller, and S. Fidler, "Extracting triangular 3D models, materials, and lighting from images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[18] T. Shen, J. Gao, K. Yin, M.-Y. Liu, and S. Fidler, "Deep marching tetrahedra: a hybrid representation for high-resolution 3D shape synthesis," 2021.

[19] T. Shen, J. Munkberg, J. Hasselgren, K. Yin, A. Lefohn, J. Gao, and S. Fidler, "Flexible isosurface extraction for gradient-based mesh optimization," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.

[20] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[21] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[22] M. Deitke, D. Schwenk, J. Salvador, L. Weihs, O. Michel, E. VanderBilt, L. Schmidt, K. Ehsani, A. Kembhavi, and A. Farhadi, "Objaverse: A universe of annotated 3D objects," 2023.

[23] L. Downs, A. Francis, N. Koenig, B. Kinman, R. Hickman, K. Reymann, T. B. McHugh, and V. Vanhoucke, "Google scanned objects: A high-quality dataset of 3D scanned household items," in *International Conference on Robotics and Automation (ICRA)*, 2022.

[24] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun, "Tanks and temples: Benchmarking large-scale scene reconstruction," *ACM Transactions on Graphics*, vol. 36, no. 4, 2017.

[25] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, "U2-Net: Going deeper with nested U-structure for salient object detection," *Pattern Recognition*, vol. 106, p. 107404, 2020.

[26] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything V2," 2024.

[27] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104–4113.

[28] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.

[29] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," 2024.


input
3DFUN AI (ours)
TSDF fusion
visual hull

input → U^2-Net matte → Depth Anything V2 → thickness field → SDF → marching cubes → Taubin smooth → planar UV → albedo + PBR bake → GLB/OBJ/PLY

Fig. 2. The reconstruction pipeline. Only the two ONNX graphs are learned; the volumetric formulation, the mesher and the texture bake are analytic, which is what allows the whole path to run on a CPU.

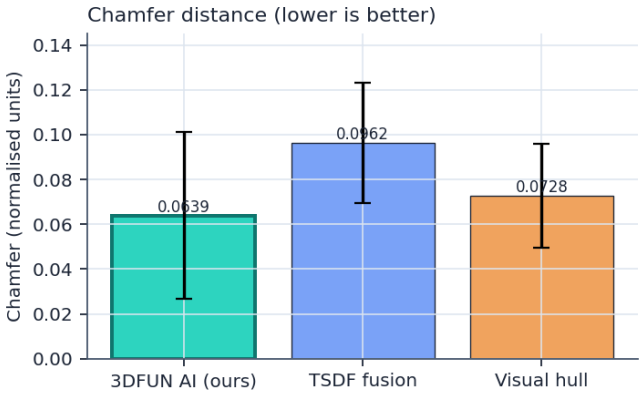


Fig. 3. Chamfer distance per method over the scenes with reference meshes (mean, whiskers are one standard deviation over repetitions). Lower is better.

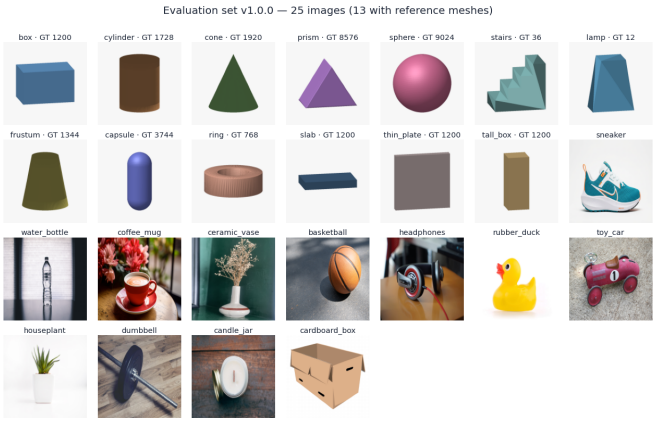


Fig. 4. The evaluation set. Synthetic scenes are labelled with the triangle count of their reference mesh; photographs carry no metric ground truth and are evaluated on the metrics that do not need one.

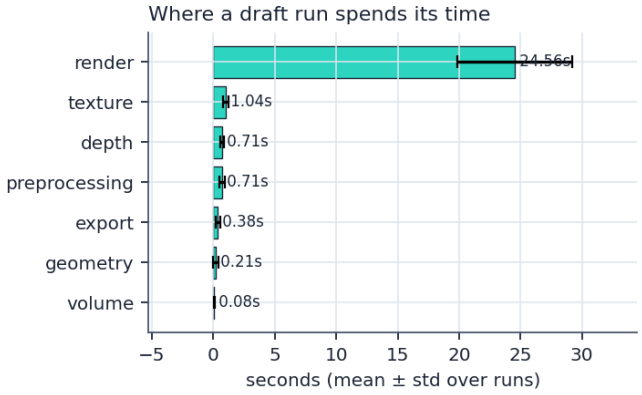


Fig. 5. Where a draft run spends its time. Reconstruction (depth, volume, geometry, texture) accounts for a small fraction; preview and turntable rendering dominate on a CPU.